

Retail Commerce Operations – 20 Spark Solutions

Simple, assessment-friendly PySpark solutions based on data/commerce.csv.

1) define_schema()

```
def define_schema():
    return StructType([
        StructField("txn_id", StringType(), True),
        StructField("order_date", StringType(), True),
        StructField("customer_id", StringType(), True),
        StructField("region", StringType(), True),
        StructField("channel", StringType(), True),
        StructField("category", StringType(), True),
        StructField("product_id", StringType(), True),
        StructField("quantity", IntegerType(), True),
        StructField("unit_price", DoubleType(), True),
        StructField("discount_rate", DoubleType(), True),
        StructField("returned", StringType(), True),
        StructField("ship_date", StringType(), True),
        StructField("delivery_date", StringType(), True)
    ])
```

2) load_data()

```
def load_data(spark, path, schema):
    return spark.read.csv(
        path,
        header=True,
        schema=schema
    )
```

3) parse_dates()

```
def parse_dates(df):
    return (
        df.withColumn("order_date", to_date("order_date"))
        .withColumn("ship_date", to_date("ship_date"))
        .withColumn("delivery_date", to_date("delivery_date"))
    )
```

4) add_gross_amount()

```
def add_gross_amount(df):
    return df.withColumn(
        "gross_amount",
        col("quantity") * col("unit_price")
    )
```

5) add_net_amount()

```
def add_net_amount(df):
    if "gross_amount" not in df.columns:
        df = add_gross_amount(df)

    return df.withColumn(
        "net_amount",
        col("gross_amount") * (1 - col("discount_rate"))
    )
```

6) add_delivery_days()

```
def add_delivery_days(df):
    return df.withColumn(
        "delivery_days",
        datediff(col("delivery_date"), col("ship_date"))
    )
```

7) flag_on_time_delivery()

```
def flag_on_time_delivery(df, max_days):
    if "delivery_days" not in df.columns:
        df = add_delivery_days(df)

    return df.withColumn(
        "is_on_time",
        col("delivery_days") <= max_days
    )
```

8) filter_returned_orders()

```
def filter_returned_orders(df):
    return df.filter(
        col("returned") == "Y"
    )
```

9) filter_by_region()

```
def filter_by_region(df, region):
    return df.filter(
        col("region") == region
    )
```

10) top_n_customers_by_spend()

```
def top_n_customers_by_spend(df, n):
    df = add_net_amount(df)

    return (
        df.groupBy("customer_id")
        .agg(
            sum("net_amount").alias("total_spend")
        )
        .orderBy(
            col("total_spend").desc(),
            col("customer_id").asc()
        )
        .limit(n)
    )
```

11) revenue_by_category()

```
def revenue_by_category(df):
    df = add_net_amount(df)

    return (
        df.groupBy("category")
        .agg(
            sum("net_amount").alias("total_revenue")
        )
    )
```

)

12) top_category_by_revenue()

```
def top_category_by_revenue(df):
    df = add_net_amount(df)

    result = (
        df.groupBy("category")
        .agg(
            sum("net_amount").alias("total_revenue")
        )
        .orderBy(
            col("total_revenue").desc(),
            col("category").asc()
        )
        .limit(1)
        .collect()
    )

    return result[0]["category"]
```

13) avg_discount_by_channel()

```
def avg_discount_by_channel(df):
    return (
        df.groupBy("channel")
        .agg(
            avg("discount_rate").alias("avg_discount")
        )
    )
```

14) daily_revenue_trend()

```
def daily_revenue_trend(df):
    df = add_net_amount(df)

    return (
        df.groupBy("order_date")
        .agg(
            sum("net_amount").alias("daily_revenue")
        )
        .orderBy(
            col("order_date").asc()
        )
    )
```

15) high_value_orders()

```
def high_value_orders(df, threshold):
    df = add_net_amount(df)

    return df.filter(
        col("net_amount") > threshold
    )
```

16) count_late_deliveries()

```
def count_late_deliveries(df, max_days):
    df = add_delivery_days(df)

    return (
```

```

        df.filter(
            col("delivery_days") > max_days
        )
        .count()
    )

```

17) return_rate_by_category()

```

def return_rate_by_category(df):
    return (
        df.groupBy("category")
        .agg(
            count("*").alias("total_cnt"),
            sum(
                when(col("returned") == "Y", 1)
                .otherwise(0)
            ).alias("returned_cnt")
        )
        .withColumn(
            "return_rate",
            col("returned_cnt") / col("total_cnt")
        )
    )

```

18) best_selling_product()

```

def best_selling_product(df):
    result = (
        df.groupBy("product_id")
        .agg(
            sum("quantity").alias("total_qty")
        )
        .orderBy(
            col("total_qty").desc(),
            col("product_id").asc()
        )
        .limit(1)
        .collect()
    )

    return (
        result[0]["product_id"],
        result[0]["total_qty"]
    )

```

19) list_channels()

```

def list_channels(df):
    df = df.select("channel").distinct().collect()

    channels = []

    for row in df:
        channels.append(row["channel"])

    channels = sorted(channels)

    return channels

```

20) orders_in_date_range()

```

def orders_in_date_range(df, start, end):
    return df.filter(
        col("order_date").between(

```

```
        to_date(lit(start)),  
        to_date(lit(end))  
    )  
)
```